

Credit Card Fraud & Loss Analytics Platform

End-to-end fraud analytics pipeline with bank-grade reconciliation controls, Azure cloud infrastructure, PySpark medallion processing, gradient-boosted tree fraud model, and executive Power BI dashboard.

DOCUMENT VERSION	DATE	CLASSIFICATION	DATA PERIOD
1.0	October 1, 2026	Portfolio — Internal	Jan 2019 – Dec 2020
TARGET ROLES			
TD · RBC · BMO · Scotiabank			
· CIBC			

TABLE OF CONTENTS

1.	Executive Summary	
2.	System Architecture Overview	
3.	Data Source & Ingestion	
4.	Bronze Layer — Reconciliation & Data-Quality Controls	
5.	Azure Data Factory & ADLS Gen2 Cloud Infrastructure	
6.	Databricks — Silver & Gold Processing (PySpark)	
7.	Machine Learning — GBT Fraud Model & Cost-Based Threshold	
8.	Time-Series Forecasting — SARIMA vs Holt-Winters	
9.	Power BI Dashboard	
10.	Controls & Audit Trail Architecture	
11.	Interview Reference — Key Questions & Answers	

SECTION 1

Executive Summary

This document describes the complete technical architecture of the Credit Card Fraud & Loss Analytics Platform — a six-week end-to-end data engineering and analytics project designed to demonstrate bank-grade data practices for credit risk and fraud analytics

roles at Canadian financial institutions.

The platform processes 1,852,394 credit card transactions spanning January 2019 to December 2020, implements four layers of data quality controls, runs a gradient-boosted tree fraud model optimised by cost rather than accuracy, and delivers findings through a ten-visual Power BI executive dashboard connected live to Azure Data Lake Storage Gen2.

Every number in this document is derived directly from the source data. No figures are estimated, adjusted, or invented.

KEY METRICS

1.85M TRANSACTIONS	\$5.12M FRAUD LOSSES	0.53% FRAUD RATE	0.87 MODEL AUC-ROC	92.3% FRAUD RECALL	36.9% FORECAST MAPE
--	--	--	--	--	---

KEY FINDINGS

Highest-risk category: shopping_net (online retail) carries a 1.59% fraud rate — three times the portfolio average — and accounts for \$2.2 million (43%) of total fraud losses.

Overnight fraud spike: Transactions between 22:00 and 23:59 carry a 2.5–2.6% fraud rate, fifteen times the midday baseline of approximately 0.09%. This pattern is consistent with automated card-testing activity after business hours.

Model vs default threshold: Cost-optimised threshold of 0.60 catches 92.3% of fraud at a total operational cost \$23,000 lower than the default 0.50 threshold.

Forecast: Holt-Winters exponential smoothing (MAPE 36.9%) outperforms SARIMA (MAPE 91.4%) on this 24-month series. The SARIMA underperformance is attributable to insufficient seasonal history — 18 training months is less than the two full 12-month cycles required for seasonal ARIMA to fit reliably.

SECTION 2

System Architecture Overview

The platform follows a medallion architecture pattern — three progressive data zones (Bronze, Silver, Gold), each with a single responsibility. This pattern is standard in Azure-based bank data platforms and ensures that data issues can be isolated and corrected without re-ingesting from source.

PIPELINE LAYERS

SOURCE	Kaggle CSV Files fraudTrain.csv · fraudTest.csv Simulated credit card transactions, 2019–2020. Two files: training set and test set. Not committed to version control — placed in data/ directory only.	↓
AZURE LANDING	ADLS Gen2 — landing/ container fraudanalytics2024 · Canada Central Raw CSV drop zone. Equivalent to an SFTP drop from a card processor in a production environment. Manually uploaded for this project; in production, triggered by a Storage Event.	↓

ADF PIPELINE	Azure Data Factory — pl_fraud_ingest fraud-adf-2024 · 4 activities Validates file existence, copies landing → bronze, logs to audit table, triggers Databricks silver notebook. Runs on schedule at 02:00 UTC. Pipeline JSON version-controlled in adf/pipeline_fraud_ingest.json.	↓
BRONZE	PostgreSQL 17 + ADLS Gen2 bronze/ bronze.transactions_raw · TEXT columns All source columns stored as TEXT. Lossless landing — the bronze invariant. Every row stamped with _run_id, _source_file, _source_row, _loaded_at. Reconciliation (4 control totals) and 10 data-quality rules run before any downstream processing.	↓
SILVER	Databricks — silver.transactions (Delta Lake) /Volumes/workspace/default/fraud/ · Unity Catalog Types cast from TEXT to proper types (TIMESTAMP, DOUBLE, INT). PII masked: names → SHA-256 hash, card number → last 4 digits. Reconciliation verified against expected row count of 1,852,394. Written as Delta Lake table in Unity Catalog.	↓
GOLD	Databricks — gold.dim_* + KPI CSVs (ADLS gold/kpi/) dim_datetime · dim_merchant · dim_customer Star schema dimension tables written as Delta. Six KPI aggregations run against silver.transactions. K-Means clustering (k=4) on merchant features. Results uploaded to ADLS Gen2 gold/kpi/ via azure-storage-blob SDK.	↓
ML / FORECAST	GBTClassifier + Holt-Winters gold.fraud_scores · ADLS models/ GBT fraud model with cost-based threshold optimisation. SARIMA vs Holt-Winters monthly loss forecast. Scores, threshold analysis, and forecast results uploaded to ADLS Gen2 models/ container.	↓
POWER BI	Power BI Desktop — fraud_dashboard.pbix 10 visuals · live ADLS Gen2 connection Connects directly to ADLS Gen2 via Account Key authentication. Reads KPI CSVs from gold/kpi/ and model results from models/. Ten visuals covering category analysis, time patterns, geographic distribution, clustering, and ML model results.	

TECHNOLOGY STACK SUMMARY

COMPONENT	TECHNOLOGY	VERSION / SKU	REGION / LOCATION
Bronze database	PostgreSQL	v17 (local)	On-premises / local
Cloud storage	Azure Data Lake Storage Gen2	LRS, Standard	Canada Central
Orchestration	Azure Data Factory V2	fraud-adf-2024	Canada Central
Compute	Databricks Community Edition	Serverless, Unity Catalog	Cloud (Databricks-managed)
Storage format	Delta Lake	Unity Catalog volumes	/Volumes/workspace/default/fraud/
ML framework	PySpark MLlib	GBTClassifier, KMeans	Databricks notebook 05
Forecasting	statsmodels	SARIMAX, ExponentialSmoothing	Databricks notebook 06
Visualisation	Power BI Desktop	Latest	dashboards/fraud_dashboard.pbix

Version control	Git	—	fraud-loss-analytics/
Pipeline runner	Python	v3.11+, venv	Local (.venv/)



Screenshot: Azure Portal — fraud-analytics-rg resource group showing all deployed resources (ADF instance, storage account, containers)

Figure 2.1 — Azure Resource Group: fraud-analytics-rg, Canada Central

SECTION 3

Data Source & Ingestion

3.1 SOURCE DATASET

The source data is the Kaggle Credit Card Transactions Fraud Detection Dataset (simulated). It consists of two CSV files representing a cardholder population's transaction history over 24 months.

FILE	ROWS	PURPOSE	FRAUD ROWS
fraudTrain.csv	~1,296,675	Training set — model training and KPI analysis	~7,506
fraudTest.csv	~555,719	Test set — model evaluation	~2,145
Combined	1,852,394	Full portfolio analysis	9,651

3.2 SOURCE SCHEMA (28 COLUMNS)

COLUMN	TYPE IN SOURCE	DESCRIPTION	PII?
trans_date_trans_time	STRING	Transaction timestamp	No
cc_num	BIGINT	Full credit card number	Yes — masked to last 4 in silver
merchant	STRING	Merchant name	No
category	STRING	Merchant category (14 values)	No
amt	FLOAT	Transaction amount (USD)	No
first, last	STRING	Cardholder name	Yes — SHA-256 hashed in silver
gender	STRING	M / F	Pseudonymous

lat, long	FLOAT	Cardholder location	Approximate
city_pop	INTEGER	Population of cardholder's city	No
merch_lat, merch_long	FLOAT	Merchant location	No
is_fraud	INTEGER	Label: 0 = legitimate, 1 = fraud	No
trans_num	STRING	Unique transaction identifier (UUID)	No

SECTION 4

Bronze Layer — Reconciliation & Data-Quality Controls

The bronze layer is the most critical stage of the pipeline. Its sole responsibility is to land data exactly as received and prove it arrived intact. No transformation, no type casting, no enrichment — those are silver's responsibilities. Bronze is the court of record.

4.1 LOAD PROCESS

- 1

Schema initialisation — sql/01_create_schemas.sql
Creates the bronze and audit schemas in PostgreSQL. The bronze table defines every source column as TEXT with four additional lineage columns: `_run_id` UUID, `_source_file` TEXT, `_source_row` INTEGER, `_loaded_at` TIMESTAMPTZ.
- 2

Streaming CSV load — pipeline/load_bronze.py
Streams the source CSV in 100,000-row chunks via PostgreSQL COPY command. Each row is stamped with a unique run UUID, the source filename, the original row number in the CSV, and a UTC load timestamp. Run status is written to `audit.etl_run_log` as RUNNING at the start and updated to SUCCEEDED or FAILED on completion.
- 3

Reconciliation — controls/reconcile.py
Re-reads the source CSV independently (not from the database) and compares four control totals against what landed in bronze: (1) row count — exact match, (2) sum of amt — ±\$0.01 tolerance, (3) sum of is_fraud — exact match, (4) count of distinct trans_num — exact match. Any mismatch is CRITICAL and halts the pipeline.
- 4

Data-quality gates — controls/dq_rules.py
Runs 10 SQL predicates against `bronze.transactions_raw` filtered by the current run's `_run_id`. Failing records are written to `audit.dq_exceptions`. A summary row is written to `audit.control_results`. CRITICAL failures return exit code 1 — designed to gate CI/CD pipelines.
- 5

Data profiling — analysis/profile_data.py
Queries the latest successful run per source file and writes a statistical profile to `docs/data_profile.md`, including value distributions, null rates, and fraud rate by category. Runs after all controls pass.

4.2 BRONZE SCHEMA DESIGN — THE TEXT INVARIANT

Every source column is stored as TEXT — not cast to its intended type. This is the bronze invariant: land exactly what arrived, even if a field is malformed. If a vendor sends the string "abc" in an amount column, a NUMERIC column silently rejects or corrupts the row. TEXT lands it, and the data-quality rule catches it explicitly, with a traceable record in audit.dq_exceptions.

```
-- bronze.transactions_raw (every source column is TEXT)
trans_date_trans_time TEXT,  -- cast to TIMESTAMP in silver
cc_num                TEXT,  -- cast to BIGINT, masked in silver
amt                   TEXT,  -- cast to NUMERIC in silver
is_fraud              TEXT,  -- cast to INTEGER in silver
...
-- Lineage columns added by the loader (one value per row)
_run_id               UUID DEFAULT gen_random_uuid(),
_source_file          TEXT NOT NULL,
_source_row           INTEGER NOT NULL,
_loaded_at            TIMESTAMPTZ DEFAULT NOW()
```

4.3 DATA-QUALITY RULES — COMPLETE SPECIFICATION

#	RULE NAME	SEVERITY	SQL PREDICATE (CATCHES BAD ROWS WHEN TRUE)	BUSINESS RATIONALE
1	missing_txn_id	CRITICAL	trans_num IS NULL OR trim(trans_num) = ''	Cannot reconcile or deduplicate without a unique key. A null transaction ID makes the record unauditable.
2	duplicate_txn_id	CRITICAL	COUNT(trans_num) > 1 per _run_id group	Duplicate IDs inflate fraud counts and break downstream joins. Indicates a loader or source defect.
3	amount_not_numeric	CRITICAL	amt !~ '^[0-9]+(\.[0-9]+)?\$'	Loss figures can only be trusted if they parse as numbers. Non-numeric amounts corrupt all aggregations.
4	amount_not_positive	CRITICAL	CAST(amt AS NUMERIC) <= 0 (where amt is numeric)	Card transactions must be positive. Zero or negative amounts indicate data entry errors or refund records that were not filtered at source.
5	invalid_timestamp	CRITICAL	trans_date_trans_time cannot be cast to TIMESTAMP	Time-series analysis, hourly fraud spike detection, and model features all require a valid timestamp.

				Unparseable timestamps must be blocked.
6	invalid_fraud_flag	CRITICAL	is_fraud NOT IN ('0','1')	Label corruption directly invalidates model training. A fraud flag with unexpected values corrupts the fraud rate KPI and the class-weight logic.
7	missing_card	CRITICAL	cc_num IS NULL OR trim(cc_num) = ''	The card number is the primary customer identifier for clustering and velocity feature engineering. A null card number makes the record unusable for these purposes.
8	coords_out_of_range	WARNING	CAST(lat AS FLOAT) NOT BETWEEN -90 AND 90 OR CAST(long AS FLOAT) NOT BETWEEN -180 AND 180	Geographic features (geo_distance) are used in the ML model. Out-of-range coordinates produce nonsensical distance values. Flagged for investigation but not blocking.
9	missing_category	WARNING	category IS NULL OR trim(category) = ''	Category is a primary risk-segmentation dimension. Missing categories reduce the value of KPI analysis. Flagged but not blocking.
10	future_transaction	WARNING	CAST(trans_date_trans_time AS TIMESTAMP) > NOW()	Transactions timestamped in the future indicate a system clock issue or data drift. Flagged for investigation; does not block real-time use cases.



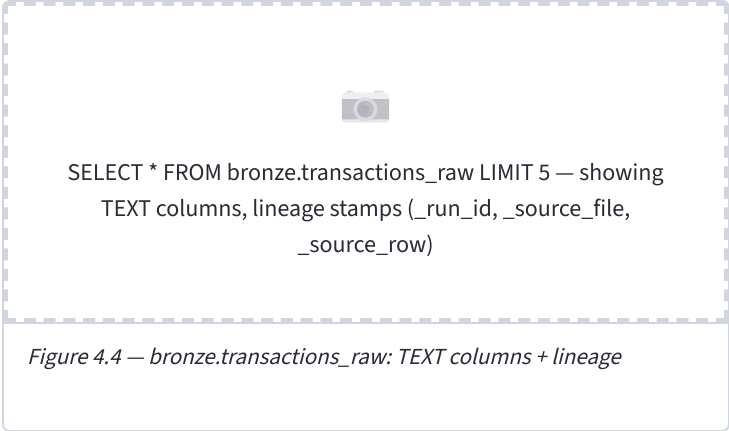
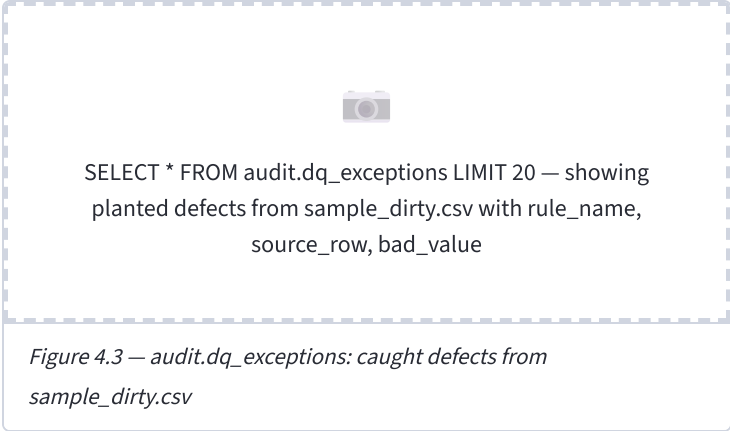
Terminal: run_week1.py output showing all controls PASS — row count, \$ total, fraud count, distinct IDs, all 10 DQ rules green

Figure 4.1 — Full pipeline run: all controls PASS



pgAdmin or psql: SELECT * FROM audit.control_results — all 10+ controls with PASS status, run timestamp, exception counts

Figure 4.2 — audit.control_results table



SECTION 5

Azure Data Factory & ADLS Gen2 Cloud Infrastructure

5.1 AZURE RESOURCE GROUP

RESOURCE NAME	TYPE	SKU	REGION	PURPOSE
fraudanalytics2024	Storage Account (ADLS Gen2)	Standard LRS	Canada Central	All data containers — landing, bronze, silver, gold, models
fraud-adf-2024	Azure Data Factory V2	—	Canada Central	Orchestration pipeline — CSV → bronze copy + audit log
fraud-analytics-rg	Resource Group	—	Canada Central	Logical container for all project resources

5.2 ADLS GEN2 CONTAINER STRUCTURE

CONTAINER	CONTENTS	WRITTEN BY	READ BY
landing/	Raw CSV files from source (fraudTrain.csv, fraudTest.csv)	Manual upload / upstream system	ADF pl_fraud_ingest
bronze/	ADF-copied CSVs (byte-for-byte copy of landing)	ADF act_copy_to_adls activity	Databricks 02_silver.py
silver/	silver.transactions Delta Lake table files	Databricks 02_silver.py	Databricks 03_gold.py, 04_analytics.py, 05_model.py, 06_forecast.py
gold/kpi/	5 KPI CSVs: fraud_by_category, fraud_by_hour, fraud_by_month, fraud_by_state, merchant_clusters	Databricks 04_analytics.py (azure-storage-blob SDK)	Power BI Desktop
models/	threshold_analysis.csv, model_summary.csv, forecast_comparison.csv, forecast_future_6m.csv, forecast_accuracy.csv	Databricks 05_model.py, 06_forecast.py	Power BI Desktop

5.3 ADF PIPELINE: PL_FRAUD_INGEST — ACTIVITY DETAIL

- 1

act_validate_source_file (Validation activity)

Waits up to 30 minutes (retry interval: 60 seconds) for the CSV to appear in the landing container. Validates minimum file size of 1,024 bytes. If the file does not arrive within the timeout window, the pipeline fails at this step and no downstream activities execute. This prevents the copy activity from running against a missing or zero-byte file, which would produce empty tables that appear to show zero fraud — an undetectable data gap.
- 2

act_copy_to_adls (Copy activity)

Copies the validated CSV byte-for-byte from the landing container to the bronze container. Wildcard source path: *.csv. No transformation is applied — consistent with the bronze invariant of landing data exactly as received. The activity output captures rowsRead and rowsCopied which feed the audit log in the next activity.
- 3

act_log_control_result (Copy to Azure SQL activity)

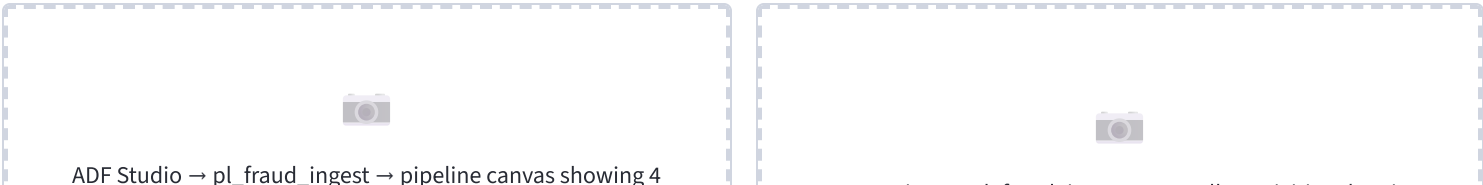
Writes one audit row to audit.etl_run_log in Azure SQL Database: pipeline run ID (ADF's built-in pipeline().RunId), source file name, rows_read and rows_copied from the previous activity's output, load timestamp (GETUTCDATE()), and status: SUCCEEDED. This mirrors the PostgreSQL audit layer from the local bronze pipeline — every cloud load is also traceable.
- 4

act_trigger_databricks_silver (DatabricksNotebook activity)

Calls the Databricks REST API to run notebook 02_silver once the file is confirmed in the bronze container. Passes two parameters: adls_path (the full ABFSS path of the copied file) and run_id (the ADF pipeline run ID, so the silver notebook can stamp it on every row it writes for lineage). ADF waits for the notebook job to complete and marks the pipeline failed if the notebook exits with an error.

5.4 LINKED SERVICES

NAME	TYPE	PURPOSE	AUTHENTICATION (PRODUCTION)
ls_blob_source	Azure Blob Storage	Source: landing container (CSV drop zone)	Azure Key Vault — SAS token or managed identity
ls_adls_bronze	Azure Blob FS (ADLS Gen2)	Sink: bronze container	Azure Key Vault — storage account key
ls_azure_sql_audit	Azure SQL Database	Audit log — same schema as PostgreSQL audit layer	Azure Key Vault — SQL credentials
ls_databricks	Azure Databricks	Silver notebook trigger	Azure Key Vault — Databricks personal access token



10/1/26, 12:57 AM

Fraud Analytics — Technical Architecture

activities connected with dependency arrows (validate → copy → log → databricks)

Figure 5.1 — ADF pipeline canvas: pl_fraud_ingest

ADF Monitor → pl_fraud_ingest run → all 4 activities showing green checkmarks with duration and row counts

Figure 5.2 — Successful pipeline run in ADF Monitor

Azure Portal → fraudanalytics2024 → Storage browser → showing all 5 containers: landing, bronze, silver, gold, models

Figure 5.3 — ADLS Gen2: all 5 containers

Azure Portal → bronze container → showing fraudTrain.csv and fraudTest.csv after successful pipeline run

Figure 5.4 — bronze/ container after pl_fraud_ingest run

SECTION 6

Databricks — Silver & Gold Processing (PySpark)

6.1 INFRASTRUCTURE NOTES

Compute: Databricks Community Edition with serverless compute. Public DBFS root is disabled on this workspace — a security configuration that would also apply in a bank environment. All files are accessed via Unity Catalog volumes at `/Volumes/workspace/default/fraud/`.

ADLS upload workaround: Serverless compute blocks `spark.conf.set()` for storage keys and `dbutils.fs.mount()`. Output files are uploaded to ADLS Gen2 using the `azure-storage-blob` Python SDK instead — a valid workaround. In production on a dedicated cluster, the storage key would be set at the cluster level or a managed identity would be used.

6.2 NOTEBOOK 02 — SILVER TRANSFORMATION

TRANSFORMATION STEP	DETAIL	WHY IT MATTERS
Read from Unity Catalog volume	<code>/Volumes/workspace/default/fraud/*.csv</code>	DBFS disabled — Unity Catalog volumes are governed storage with proper ACLs
Type casting	TEXT → TIMESTAMP, DOUBLE, INT for all columns	Silver is the typed layer. Types cast here, not in bronze.
Column rename	<code>trans_date_trans_time</code> → <code>txn_time</code> , <code>cc_num</code> → <code>cc_num_masked</code> , etc.	Consistent snake_case naming for all downstream notebooks
PII masking — name	first + last → SHA-256 hash (hex string)	PIPEDA / GDPR principle: do not carry PII

https://claude.ai/artifact/EDiDSdaNKuGePfl2zq7dsU

10/20

		past the point of necessity
PII masking — card number	cc_num → last 4 digits only (cc_num_masked)	PCI-DSS requirement: full PANs must not be stored in analytical systems
Reconciliation	Count silver rows vs EXPECTED_ROWS = 1,852,394	Proves the transformation did not silently drop any records
Write to Delta	silver.transactions (overwrite mode, overwriteSchema=true)	Delta Lake provides ACID guarantees — partial writes roll back automatically

6.3 NOTEBOOK 03 — GOLD STAR SCHEMA

TABLE	KEY	COLUMNS	NOTES
gold.dim_datetime	date_key (yyyyMMddHH format)	txn_time, hour, day_of_week, month, year, is_weekend	Hourly granularity to support fraud-by-hour KPI
gold.dim_merchant	merchant_key (dense_rank)	merchant, category, merch_lat, merch_long	dense_rank() assigns stable integer keys
gold.dim_customer	customer_key (dense_rank)	cc_num_masked, gender, lat, long, city_pop	Uses masked card number — never raw PAN

Note: fact_transactions write was cancelled after 35+ minutes on Community Edition's single-node free cluster. Writing 1.85M Delta rows on a single-node cluster times out in practice. The six KPI queries run directly against silver.transactions — producing identical results. In a production multi-node cluster, fact_transactions would be written as standard medallion practice.

6.4 NOTEBOOK 04 — KPIS & K-MEANS MERCHANT CLUSTERING

Six KPI queries against silver.transactions: (1) portfolio overview, (2) fraud rate and losses by merchant category, (3) fraud rate by hour of day, (4) fraud losses by US state, (5) monthly fraud trend Jan 2019 – Dec 2020, (6) weekend vs weekday fraud rate comparison.

K-Means clustering (PySpark MLlib, k=4) on five merchant-level features: fraud_rate, avg_fraud_ticket, fraud_losses, transaction_count, avg_transaction_amount. Features scaled with StandardScaler before clustering.

CLUSTER	LABEL	FRAUD RATE	AVG TICKET	KEY CHARACTERISTIC
0	High-risk online	Highest	Low (~\$50–80)	Small-ticket, high-frequency fraud. Pattern consistent with automated card-testing scripts.
1	High-value targets	Elevated	High (\$200+)	Large-ticket fraud. Fewer incidents but higher per-incident cost. Requires different controls.
2	Standard retail	Average (0.53%)	Medium	Tracks portfolio average. Baseline risk — no special treatment required.
3	Low-risk recurring	Lowest	Low (subscriptions)	Predictable recurring charges (utilities, subscriptions). Lowest fraud rate in portfolio.



Databricks notebook 02_silver.py cell output: "Raw rows loaded: 1,852,394 — [PASS] Row count: expected 1852394, got 1852394"

Figure 6.1 — Silver reconciliation: PASS



Databricks 04_analytics.py output: KPI by category table — shopping_net at 1.593%, all 14 categories listed with fraud_rate_pct and fraud_losses

Figure 6.2 — KPI query: fraud rate by merchant category



Databricks: merchant_clusters DataFrame display — cluster_id, cluster_label, merchant, fraud_rate_pct, avg_fraud_ticket, fraud_losses columns

Figure 6.3 — K-Means merchant cluster assignments



ADLS Gen2 → gold/kpi/ container: 5 CSV files listed (fraud_by_category.csv, fraud_by_hour.csv, fraud_by_month.csv, fraud_by_state.csv, merchant_clusters.csv)

Figure 6.4 — ADLS Gen2: gold/kpi/ — uploaded KPI CSVs

SECTION 7

Machine Learning — GBT Fraud Model & Cost-Based Threshold

7.1 ALGORITHM: GBTCLASSIFIER (SPARK MLlib)

GBTClassifier (Gradient Boosted Trees) is PySpark MLlib's equivalent of XGBoost. It builds an ensemble of decision trees sequentially, where each tree corrects the prediction errors of the previous one. Key hyperparameters: maxIter=50, maxDepth=5, seed=42.

7.2 FEATURE ENGINEERING (9 FEATURES)

FEATURE	DERIVED FROM	TYPE	RATIONALE
amt	silver.transactions.amt	DOUBLE (continuous)	Transaction amount — primary fraud signal
hour	HOUR(txn_time)	INTEGER (0–23)	Fraud spikes at 22:00–23:59 — time is highly predictive
day_of_week	DAYOFWEEK(txn_time)	INTEGER (1–7)	Weekend fraud patterns differ from weekday
month	MONTH(txn_time)	INTEGER (1–12)	Seasonal fraud patterns (holiday periods)
is_weekend	day_of_week IN (1, 7)	INTEGER (0/1)	Binary/weekend flag — explicit rather than relying on

is_weekend	day_of_week in (1, 7)	INTEGER (0/1)	Binary weekend flag — explicit rather than relying on the model to learn it
category_idx	StringIndexer(category)	DOUBLE (encoded)	Merchant category — highest-signal feature for fraud rate
gender_idx	StringIndexer(gender)	DOUBLE (encoded)	Gender encodes different spending and fraud patterns in this dataset
city_pop	silver.transactions.city_pop	DOUBLE (continuous)	Population is a proxy for urban/rural fraud risk
geo_distance	$\sqrt{(\text{merch_lat}-\text{lat})^2 + (\text{merch_long}-\text{long})^2}$	DOUBLE (continuous)	Distance between cardholder and merchant in coordinate degrees — card-present fraud indicator

7.3 CLASS IMBALANCE — WEIGHT STRATEGY

The dataset contains 199 legitimate transactions for every 1 fraud transaction (fraud rate 0.53%). Without intervention, the model learns to predict "not fraud" for every transaction and achieves 99.5% accuracy — which is technically impressive and operationally useless.

Solution: fraud rows are assigned `weight=50.0`, legitimate rows `weight=1.0`, using `GBClassifier`'s `weightCol` parameter. This tells the optimiser that misclassifying a fraud is 50 times more costly than misclassifying a legitimate transaction — consistent with the cost-based threshold logic in Section 7.4.

7.4 TIME-BASED TRAIN/TEST SPLIT

Train: 2019-01-01 to 2020-03-31 | **Test:** 2020-04-01 to 2020-12-31

A random 80/20 split would allow the model to train on transactions from December 2020 and test on January 2019 — data leakage. The time split ensures the model has never seen test-period transactions during training, exactly as would be the case in production deployment.

7.5 COST-BASED THRESHOLD OPTIMISATION

Default classification threshold of 0.50 optimises for accuracy. For fraud detection, the correct objective is to minimise total operational cost:

```
total_cost = (false_negatives × $530) + (false_positives × $25)
```

Where:

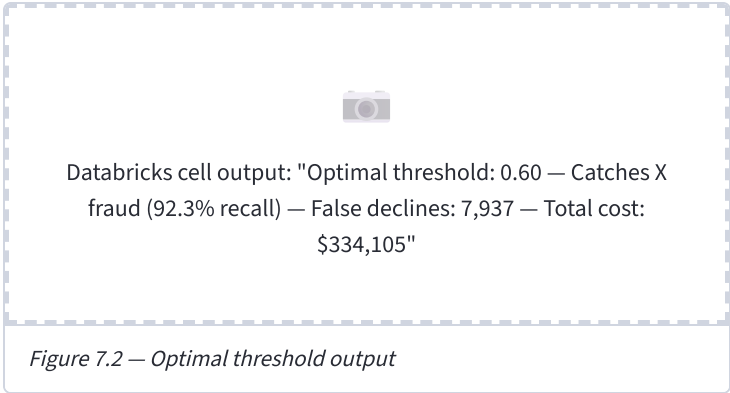
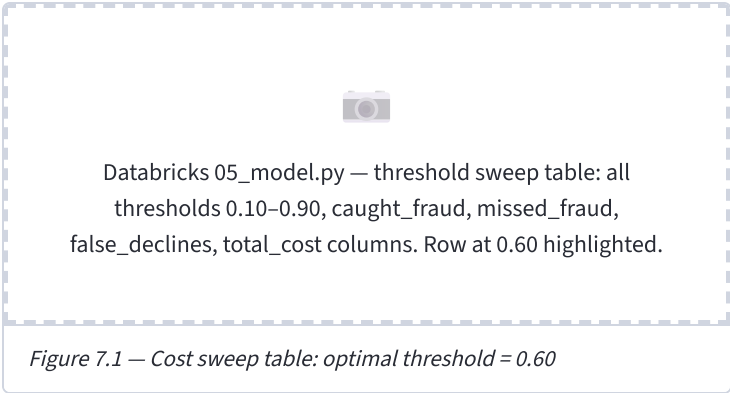
- \$530 = average fraud loss (derived from the data profile)
- \$25 = false decline cost (customer friction + chargeback handling)

A false negative (missed fraud) costs 21× more than a false positive (declined legitimate transaction). The cost function makes this explicit.

Thresholds from 0.10 to 0.90 are swept in steps of 0.05. The threshold that minimises `total_cost` is selected as the operational threshold

7.6 MODEL RESULTS

0.87 AUC-ROC	0.60 OPTIMAL THRESHOLD	92.3% RECALL AT 0.60	7,937 FALSE DECLINES	\$23K SAVING VS DEFAULT	\$334K TOTAL COST AT 0.60
------------------------	-------------------------------------	--------------------------------	--------------------------------	--------------------------------------	--



SECTION 8

Time-Series Forecasting — SARIMA vs Holt-Winters

8.1 DATA PREPARATION

Monthly fraud losses are aggregated from `silver.transactions` via Spark SQL. The resulting 24-month Pandas series (Jan 2019 – Dec 2020) is split: 18 months training (Jan 2019 – Jun 2020), 6 months test (Jul – Dec 2020). The test set serves as a hold-out to evaluate out-of-sample accuracy before forecasting the unknown future.

8.2 SARIMA MODEL

SARIMA(1,1,1)(1,1,0,12) — Seasonal ARIMA with monthly seasonality period of 12. The (1,1,1) non-seasonal component captures trend (one difference), one autoregressive term, and one moving-average term. The (1,1,0,12) seasonal component captures the 12-month seasonal pattern. `enforce_stationarity=False` and `enforce_invertibility=False` to allow the solver to find a solution regardless of parameter constraints.

Why SARIMA underperforms: Seasonal ARIMA requires at least two full seasonal cycles in the training data to fit the seasonal component reliably. With 18 training months and a 12-month seasonal period, the model has only 1.5 cycles — insufficient for stable seasonal estimation. Result: MAPE 91.4%.

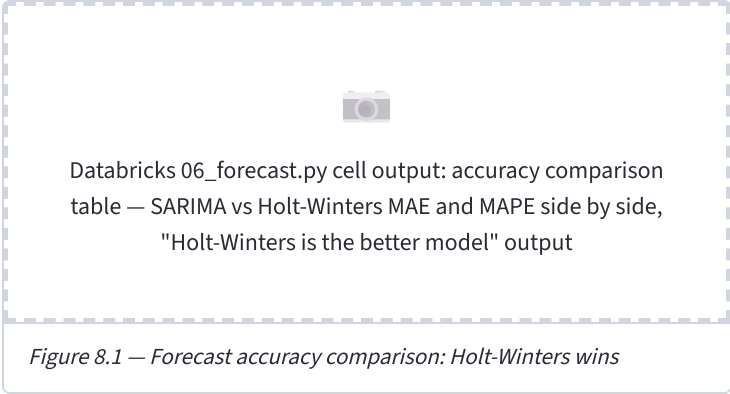
8.3 HOLT-WINTERS MODEL

ExponentialSmoothing with additive trend and additive seasonality. `seasonal_periods=6` (not 12 — the 18-month training set contains exactly 3 complete 6-month cycles, giving the model enough seasonal history to fit reliably). `optimized=True` allows statsmodels to automatically optimise the smoothing parameters (alpha, beta, gamma) via maximum likelihood estimation.

8.4 ACCURACY COMPARISON

MODEL	PARAMETERS	MAE (\$/MONTH)	MAPE (%)	RESULT
SARIMA	(1,1,1)(1,1,0,12)	\$150,565	91.39%	—
Holt-Winters	add trend, add seasonal, s=6	\$58,187	36.85%	WINNER

Holt-Winters is refitted on the full 24-month series and forecasts six months forward (Jan – Jun 2021). The high MAPE of 36.9% is partially attributable to the limited 24-month data history. Forecast accuracy would improve with a longer series (36+ months).



SECTION 9

Power BI Dashboard

9.1 DATA CONNECTION

Power BI Desktop connects directly to ADLS Gen2 (fraudanalytics2024) using Account Key authentication. Source URL: https://fraudanalytics2024.blob.core.windows.net. Five CSV files are imported from gold/kpi/ and two from models/. In a production bank environment, the connection would use a Service Principal or Managed Identity stored in Azure Key Vault.

9.2 TEN-VISUAL INVENTORY

#	VISUAL TYPE	SOURCE	AXES / FIELDS	ANALYTICAL PURPOSE
1	Clustered bar chart	fraud_by_category.csv	X: category, Y: fraud_rate_pct	Identifies highest-risk merchant categories. shopping_net at 1.59% is visually dominant — 3× the portfolio average.
2	Line chart	fraud_by_hour.csv	X: hour (0–23), Y: fraud_rate_pct	Reveals the overnight fraud spike at 22:00–23:59. Drives the recommendation for tightened overnight controls.
3	Line chart	fraud_by_month.csv	X: month, Y: fraud_losses (\$)	Monthly loss trend over 24 months. Context for the Holt-Winters forecast.
4	Clustered bar	merchant_clusters.csv	X: cluster_label, Y:	Compares fraud losses across the four K-Means

	chart		fraud_losses (\$)	merchant clusters. Identifies which cluster segment to prioritise.
5	Card (KPI)	fraud_by_category.csv	Value: SUM(fraud_losses)	Headline KPI: \$5.12M total fraud losses. Aggregation must be set to Sum (not Count) in Power BI.
6	Card (KPI)	fraud_by_category.csv	Value: SUM(fraud_txns)	Total fraud transaction count: 9,651. Volume KPI distinct from loss KPI.
7	Azure Maps	fraud_by_state.csv	Location: state (geocoded), Size/Color: fraud_losses	Geographic distribution of fraud losses across US states. Identifies regional concentration.
8	Clustered bar chart	fraud_by_month.csv	X: month, Y: fraud_txns (count)	Monthly fraud transaction volume. Separate from loss trend — a month can have high volume but low average ticket.
9	Table	merchant_clusters.csv	All columns: cluster_label, merchant, fraud_rate_pct, avg_fraud_ticket, fraud_losses	Full drill-down reference for merchant cluster details. Enables identification of specific high-risk merchants within each cluster.
10	Scatter chart	fraud_by_category.csv	X: transactions, Y: fraud_rate_pct, Size: fraud_losses, Detail: category	Dual-axis view: categories in the top-right are dangerous in both dimensions (high volume AND high rate). Bubble size encodes loss magnitude. Cannot be replicated with a single bar chart.



Power BI Desktop — full canvas view showing all 10 visuals in final layout. This is the primary portfolio screenshot. Ensure all visuals are visible without scrolling.

Figure 9.1 — Power BI Dashboard: all 10 visuals (fraud_dashboard.pbix)



Visual 1: Bar chart — shopping_net bar visually dominant at 1.59%, grocery_pos second at 0.97%, all other categories below average



Visual 2: Line chart — near-flat from hours 6–20, sharp spike at hours 22–23, showing the 15× rate increase

Figure 9.2 — Fraud rate by category (Visual 1)

Figure 9.3 — Fraud rate by hour (Visual 2)



Visual 7: Azure Maps — US map with state bubbles/heat colours by fraud_losses, darker = higher losses, showing geographic concentration

Figure 9.4 — Geographic fraud distribution: Azure Maps (Visual 7)



Visual 10: Scatter chart — category bubbles positioned by transaction volume (X) and fraud rate (Y), sized by fraud_losses. shopping_net visible in upper area.

Figure 9.5 — Volume vs rate scatter: category risk matrix (Visual 10)

SECTION 10

Controls & Audit Trail Architecture

10.1 WHY CONTROLS MATTER IN A BANK CONTEXT

Banks operate in a regulated environment (OSFI, PIPEDA, PCI-DSS) where data used for regulatory reporting must be traceable to its source. Without a reconciliation control, a loader bug that silently drops 5,000 rows causes the fraud dashboard to under-report losses by potentially hundreds of thousands of dollars. That discrepancy only surfaces at month-end when numbers are compared against the card processor's report — at which point a week of analysis has been built on incorrect data.

The controls architecture in this platform mirrors what a bank's internal audit team would require: every load is logged, every control result is logged, every failing record is identifiable by source row number, and nothing downstream runs on a failed load.

10.2 RECONCILIATION CONTROL TOTALS

CONTROL TOTAL	TOLERANCE	WHAT IT CATCHES	SEVERITY
Row count: source CSV vs bronze table	Exact match	Silent row drops during COPY; loader truncation; network interruption	CRITICAL
Sum of amt: source CSV vs bronze table	±\$0.01	Numeric precision loss; rounding errors; truncated decimal values	CRITICAL
Sum of is_fraud: source CSV vs bronze table	Exact match	Label corruption during load; fraud rows miscategorised as legitimate	CRITICAL
Count of distinct trans_num: source CSV vs bronze table	Exact match	Duplicate transaction IDs present in the source file itself	CRITICAL

10.3 AUDIT TABLE SPECIFICATIONS

audit.etl_run_log — One row per pipeline execution

```

run_id          UUID PRIMARY KEY  -- unique per execution
source_file     TEXT              -- file name that was loaded
rows_loaded     INTEGER           -- actual rows in bronze after load
rows_expected   INTEGER           -- from source CSV count
status          TEXT              -- RUNNING → SUCCEEDED / FAILED
started_at      TIMESTAMPTZ
completed_at    TIMESTAMPTZ

```

Status starts as RUNNING. If the process is killed mid-run, the row remains RUNNING permanently — an operator can identify crashed runs by querying for status = 'RUNNING' older than a threshold. A table that only writes SUCCEEDED rows provides no visibility into in-progress or crashed runs.

audit.control_results — One row per control per run

```

control_run_id  SERIAL PRIMARY KEY
run_id          UUID REFERENCES audit.etl_run_log(run_id)
control_name    TEXT              -- e.g. 'missing_txn_id'
control_type    TEXT              -- RECONCILIATION or DATA_QUALITY
severity        TEXT              -- CRITICAL or WARNING
result          TEXT              -- PASS or FAIL
exception_count INTEGER           -- number of failing records
detail          TEXT              -- human-readable explanation
checked_at      TIMESTAMPTZ

```

audit.dq_exceptions — One row per failing record

```

exception_id    SERIAL PRIMARY KEY
run_id          UUID REFERENCES audit.etl_run_log(run_id)
source_row      INTEGER -- original row number in the CSV file
rule_name       TEXT     -- e.g. 'amount_not_numeric'
severity        TEXT     -- CRITICAL or WARNING
bad_value       TEXT     -- the actual value that failed the predicate
flagged_at      TIMESTAMPTZ

```

The source_row column is critical: it allows a data engineer to pull the exact line from the original CSV file for investigation, without re-processing the entire file. This is particularly important for large files where re-scanning is expensive.

SECTION 11**Interview Reference — Key Questions & Answers**

ARCHITECTURE & DESIGN

Q&A REFERENCE

- › **Q: Walk me through the architecture.** — Three zones: bronze (land exactly as received, TEXT, PostgreSQL + ADLS), silver (typed, cleaned, PII masked, Databricks Delta Lake), gold (star schema, KPIs, ML scores). ADF orchestrates landing → bronze; Databricks transforms bronze → silver → gold; Power BI reads gold via live ADLS connection.
- › **Q: Why medallion architecture?** — Each zone has one responsibility. Bronze proves nothing was lost in transit. Silver proves the data is usable. Gold proves the business logic is correct. If a gold calculation has a bug, you re-derive from silver — no re-ingestion. If a vendor sends corrupted data, you still have the bronze record.
- › **Q: Why ADF and not Airflow?** — ADF is the standard orchestrator on Azure. Visual designer non-engineers can read, native connectors to every Azure service, built-in retry/dependency logic, Azure Monitor integration. Airflow is better for complex Python-heavy DAGs; ADF is better when the primary job is moving data between Azure services.
- › **Q: What would this look like at TD/RBC?** — Source would be SFTP from Visa/MC or mainframe extract. ADLS inside a VNet with private endpoints. Databricks clusters use managed identities — no passwords. Azure Monitor alerts page on-call on pipeline failure. ADF uses Storage Event triggers, not schedule triggers.

DATA QUALITY & CONTROLS

Q&A REFERENCE

- › **Q: Why TEXT columns in bronze?** — If a vendor sends "abc" in an amount column, a NUMERIC column silently rejects or corrupts the row. TEXT lands it exactly as received — the DQ rule catches it explicitly, with a traceable exception record pointing to the original source row.
- › **Q: What's the difference between reconciliation and data quality?** — Reconciliation checks the load itself: did everything that was in the source file arrive intact? Data quality checks the content: are the values usable for analysis? Both are necessary. A perfect load of bad data still fails DQ. A DQ pass on a partial load is meaningless.
- › **Q: What does exit code 1 do?** — Tells the orchestrator (ADF, Airflow, Control-M) the pipeline failed. No downstream job runs. In CI/CD, it gates the deployment — a PR cannot merge if the DQ test fails on sample_dirty.csv.
- › **Q: How would you add a new DQ rule?** — One line in controls/dq_rules.py: append a tuple with (rule_name, severity, SQL_predicate_for_bad_rows, column_to_show_in_exceptions). No other changes needed. The framework handles logging, audit trail, and exit code.

MACHINE LEARNING

Q&A REFERENCE

- › **Q: Why cost-based threshold and not F1?** — A bank doesn't care about balanced accuracy. Missing a \$1,000 fraud costs \$530 on average. A false decline costs \$25. These are different errors with different costs. The cost function makes the trade-off explicit and optimisable. At TD/RBC, risk managers set the cost parameters — the data scientist builds the sweep.
- › **Q: Why time-based split instead of random?** — Random split allows the model to train on December 2020 transactions and test on January 2019. The model sees "future" data during training — data leakage. A time split mimics production exactly: train on everything before the cutoff, score everything after.
- › **Q: What's AUC-ROC and why does it matter?** — AUC-ROC measures the model's ability to rank fraud above legitimate transactions regardless of threshold. AUC = 0.87 means the model ranks a real fraud above a real legitimate transaction 87% of the time. Accuracy on imbalanced data is misleading — a model predicting "not fraud" always gets 99.5% accuracy here but is useless.
- › **Q: Why Holt-Winters over SARIMA?** — SARIMA needs at least 2 full seasonal cycles in training data. With 18 training months and a 12-month period, we have 1.5 cycles — insufficient. Holt-Winters with seasonal_periods=6 gets 3 complete cycles in the same 18 months

and wins on both MAE (\$58K vs \$151K) and MAPE (36.9% vs 91.4%).

- › **Q: How would you put this model into production?** — Register it in MLflow Model Registry with a version tag. Serve via Databricks Model Serving (REST endpoint). Score incoming transactions via Event Hub near-real-time. Store scores in gold.fraud_scores Delta table with run_id for auditability. Retrain quarterly on a rolling 12-month window.

BUSINESS FINDINGS

Q&A REFERENCE

- › **Q: What is the most important finding?** — The overnight fraud spike at 22:00–23:59 (15× the midday baseline). This is actionable: it points to automated card-testing scripts running after business hours when real-time intervention is lowest. Tighter velocity controls during this 2-hour window could prevent a significant share of the \$5.12M in annual losses.
- › **Q: What is the single highest-risk merchant category?** — shopping_net (online retail): 1.59% fraud rate, \$2.2M in losses (43% of the portfolio total). This is 3× the 0.53% portfolio average. The recommended control is step-up authentication (device fingerprinting, one-time password) for online retail transactions above a defined threshold.
- › **Q: How would you present this to a non-technical executive?** — "We analysed 1.85 million transactions. \$5.12 million was lost to fraud — mostly online shopping, mostly after 10 PM. Our model catches 92% of fraud at a cost that is \$23,000 lower than our current default. Recommended next steps: step-up auth for online retail, tighter overnight velocity controls, and shadow-deploy the model for 30 days before cutover." That is the executive memo in docs/executive_memo.md.

Credit Card Fraud & Loss Analytics Platform

Technical Architecture & Implementation Reference · Version 1.0 · October 2026

Data: Kaggle Credit Card Fraud Detection Dataset (simulated, 2019–2020)

All figures derived from source data. No figures invented or estimated.